

Final Submission Documentation: Secure Password Wallet

GitHub Repository URL

[Secure Password Wallet Source Code](#)

This repository contains the complete, version-controlled source code for the Secure Password Wallet project, including the frontend application, both backend microservices, and all associated configuration files.

1. System Architecture

The **Secure Password Wallet** utilizes a modular, microservices-based architecture designed for scalability, separation of concerns, and robust security.

- **Frontend:** A Single Page Application (SPA) built with React (via Vite) and styled using custom Vanilla CSS (glassmorphism aesthetics). It serves as the user interface, managing state via the React Context API and communicating with the backend microservices using Axios.
 - **Auth Service (Node.js/Express - Port 3001):** An independent microservice responsible strictly for user identity. It handles user registration, password hashing (via `bcryptjs`), and generates JSON Web Tokens (JWT) for stateless authentication.
 - **Vault Service (Node.js/Express - Port 3002):** A dedicated microservice managing the core business logic. It handles the CRUD operations for passwords, symmetric encryption/decryption of credential payloads (using Node's native `crypto` module with `AES-256-GCM`), and evaluates password strength using the `zxcvbn` library.
 - **Database (MongoDB):** A NoSQL database used for persistent storage. Each microservice maintains logical separation within the database schema to ensure loose coupling.
-

2. Component Hierarchy

The React frontend maintains a clean, hierarchical component structure:

App (Main Entry & Router)

AuthProvider (Context API for global auth state)

Login (Public Route - User Authentication)

Register (Public Route - Account Creation)

PrivateRoute (HOC for protecting authenticated routes)

Dashboard (Main application interface)

CredentialModal (Overlay for Adding/Editing credentials)

3. API Documentation

Auth Service (<http://localhost:3001>)

1. Register User

- **Method:** POST
- **Path:** /api/auth/register
- **Request Body:**

```
{
  "username": "johndoe",
  "email": "johndoe@example.com",
  "password": "MasterPassword123!"
}
```

- **Response:** Returns the newly created user object and a JWT Bearer token.

2. Login User

- **Method:** POST
- **Path:** /api/auth/login
- **Request Body:**

```
{
  "email": "johndoe@example.com",
  "password": "MasterPassword123!"
}
```

- **Response:** Returns the authenticated user object and a JWT Bearer token.

Vault Service (`http://localhost:3002`)

Note: All Vault Service routes require an `Authorization: Bearer <token>` header.

1. Get All Credentials

- **Method:** GET
- **Path:** `/api/credentials`
- **Response:** Array of decrypted credential objects belonging to the authenticated user.

2. Add New Credential

- **Method:** POST
- **Path:** `/api/credentials`
- **Request Body:**

```
{
  "title": "Netflix",
  "username": "user@netflix.com",
  "password": "MySecretPassword",
  "url": "https://netflix.com",
  "notes": "Family account"
}
```

- **Response:** The saved (and encrypted at rest) credential object, including the computed `zxcvbn` password strength.

3. Update Credential

- **Method:** PUT
- **Path:** `/api/credentials/:id`
- **Request Body:** Same schema as Add New Credential.
- **Response:** The updated credential object.

4. Delete Credential

- **Method:** DELETE
 - **Path:** /api/credentials/:id
 - **Response:** Success confirmation message.
-

4. Database Models

The system relies on Mongoose schemas to structure the MongoDB documents.

User Schema (auth-service)

- username (String): Unique identifier, required, minimum 3 characters.
- email (String): Unique identifier, required, regex validated.
- password (String): Stored as a bcrypt hash.
- createdAt (Date): Automatically generated timestamp.

Credential Schema (vault-service)

- userId (String): The reference ID linking the credential to the authenticated user.
 - title (String): The platform or service name (e.g., "Google").
 - username (String): The login identifier for the specific platform.
 - password (String): The password payload (stored encrypted via AES-256-GCM, along with the initialization vector).
 - url (String): Optional website URL.
 - notes (String): Optional metadata.
 - passwordStrength (Object): Stores the evaluated strength score (0-4) generated by zxcvbn .
 - createdAt & updatedAt (Date): Automatically generated timestamps.
-

5. AI Usage Recap & Reflection

This project was developed through a collaborative pair-programming session with an AI Coding Assistant.

Major Prompts Used:

1. **Architectural Planning:** *"I want to build a Secure Password Wallet for my FSAD assignment... Please generate a detailed Implementation Plan and DB Schema for my*

review before starting."

2. **Project Scaffolding:** *"I approve the implementation plan. Let's proceed with MongoDB and the Password Strength Checker. Please start by scaffolding the project structure and the Auth Service first."*
3. **Execution & Setup:** *"Node is installed. Please run the npm install and scaffold the services now."*
4. **Service Management:** *"Now that MongoDB is running, please restart the Auth and Vault services so they can pick up the new database connection."*
5. **Version Control:** *"I have my Personal Access Token... Please use it to add the remote origin and push my code to the main branch now."*

Technical Challenges Resolved:

- **Environment Bootstrapping:** Initial commands for Git initialization failed because macOS Xcode Command Line Tools were missing. The AI accurately diagnosed this based on the macOS prompt behavior and paused execution while the dependencies were manually installed.
- **MongoDB Connection Issues:** Upon spinning up the microservices, Mongoose threw `ECONNREFUSED 127.0.0.1:27017` errors. The AI utilized terminal commands (`lsof -i :27017`) to determine that the local MongoDB server was not running. Once the server was manually started, the AI gracefully terminated and restarted the persistent `nodemon` processes to successfully establish the database connections.
- **GitHub Authentication Failure:** During the Git push phase, the GitHub Personal Access Token (PAT) threw a `fatal: Authentication failed` error. The AI analyzed the error logs and correctly identified that the token was either expired, revoked, or lacked the appropriate `repo` scope, advising a token regeneration or SSH alternative to successfully complete the deployment.

6. Demonstration Video

[Watch the Secure Password Wallet Demo](#)

This brief demonstration video showcases the complete user workflow, including account registration, secure login, and the real-time management of encrypted credentials within the dynamic glassmorphism interface.